

Fintech Platform Data Model

A Complete Schema, Relationship Map, and Design Rationale

Core banking, cards, loans, and investment-sync architecture for a payments platform

Prepared as a learning-oriented system design walkthrough

June 2026

Contents

1. Executive Summary
2. Critique of the Original Schema
3. Foundational Principles for Fintech Data Modeling
4. The Revised Schema (Entity-by-Entity)
5. Relationship Map & ER Diagrams
6. SQL DDL Reference Implementation
7. Architectural Patterns Worth Understanding Deeply
8. Suggested Build Roadmap
9. Glossary & Further Reading

1. Executive Summary

You asked for three things: a corrected and complete schema, a researched explanation of how real fintech systems model this data, and a comprehensive document tying it together. This document does all three, and it is written so that each decision teaches the reasoning behind it rather than just handing you a finished diagram to memorize — which matches how you've approached your other system design preparation.

The single biggest structural change in this revision is the introduction of a double-entry ledger as the source of truth for money movement, replacing the original design's habit of storing balances and ID-lists directly on parent records. Every other change in this document is in service of that one idea, or of keeping sensitive data (card numbers, passwords) out of harm's way.

What changed, at a glance:

- ACCOUNT no longer stores transaction/card/invoice lists as embedded arrays — these are derived by querying child tables on their foreign key.
- A new LEDGER_ENTRY table implements double-entry accounting: every transaction produces exactly one debit row and one credit row, and the two must always sum to zero.
- ACCOUNT.balance becomes a derived (but cached) value, rebuilt from the ledger rather than written to directly by application code.
- CARD never stores a raw card number. It stores a card_token (from a PCI-compliant processor/tokenization vault) plus a masked pan_last4 and pan_bin for display — never the CVV, ever.
- USER and ACCOUNT relationship is corrected to a clean one-to-many (one user, several accounts), removing the circular reference in the original design.
- INVESTMENTS is restructured so company_id is no longer a primary key (it can't be — many users would invest in the same company), with a new INVESTMENT_HOLDING table per user position and a separate SECURITY reference table.
- New supporting tables fintech systems need but the original list didn't have: CURRENCY (reference data), KYC_VERIFICATION (compliance), AUDIT_LOG (forensics/compliance trail), LOAN_REPAYMENT_SCHEDULE (amortization), CARD_PROVIDER, and EXTERNAL_BROKERAGE_LINK (for the brokerage-API sync you described).

Section 2 walks through exactly what was wrong with each entity in your draft and why — treat that section as the most valuable part for learning, since the goal isn't just a working schema but an understanding of which mistakes are common (and dangerous) in fintech systems specifically.

2. Critique of the Original Schema

This section goes entity by entity through your original draft. The goal isn't to be harsh about it — the instinct behind every one of these fields was reasonable, and the overall shape (User, Account, Transaction, Investments, Cards, Loans) is exactly the right set of nouns for this domain. The problems are almost all the same *kind* of problem repeated: storing derived or list-shaped data directly on a parent row instead of letting the database compute it from child rows. That single pattern, fixed once, resolves most of the issues below.

2.1 USER

Original issue: USER carries its own `account_id` field, while ACCOUNT also has `account_id` as its primary key. This creates a circular relationship — USER points to an account, and ACCOUNT doesn't point back to a user at all. There's no way to know which user actually owns a given account from the ACCOUNT side, and if a user ever has more than one account (current + savings + investment wallet, which is the norm, not the exception), the single `account_id` field on USER can't hold more than one value.

Fix: invert the direction of the foreign key.

USER should not reference ACCOUNT at all. Instead, `ACCOUNT.user_id` references `USER.user_id`. This makes it a clean one-to-many relationship: one user, many accounts. This is the same pattern as a customer having many orders, or an author having many books — the “many” side always holds the foreign key to the “one” side, never the reverse.

Secondary issue: Present-Address and permanent Address (sic) are reasonable fields, but for a fintech app handling KYC, address history sometimes needs to be versioned (regulators may ask “what address was on file when this transaction happened”). For a learning-stage project, two flat fields are fine; flag this as a future evolution rather than solving it now.

2.2 ACCOUNT

This is where most of the corrections concentrate. Walking through the original fields:

- `balance` — in the original list this wasn't even an explicit field, but it's implied. Storing balance as a column that the application increments and decrements directly is the most common source of fintech bugs: two concurrent transactions can both read the old balance, both add their delta, and one write clobbers the other (a classic race condition / lost update). The fix is to never write balance directly — it should be the sum of ledger entries for that account, optionally cached for fast reads.
- `transaction` (a list of transaction ids) — storing a list of IDs inside a row breaks First Normal Form (1NF), which says every field should hold a single atomic value, not a collection. Practically, this means you can't efficiently ask “give me all transactions over ₦10,000 this month” without pulling every account row and parsing its list. The fix: TRANSACTION already has a foreign key pointing at ACCOUNT (`sender_account_id` / `receiver_account_id`) — that foreign key is the relationship. You never need to also store the reverse list.

- Debit_Transaction / Credit_Transaction (separate lists) — same problem as above, doubled. These become unnecessary entirely once you query TRANSACTION (or better, LEDGER_ENTRY) filtered by account_id and direction.
- Cards (a list) — same problem again. CARD.account_id already creates the relationship from the other side.
- invoices (a list) — same problem again. INVOICE.account_id is sufficient.
- revenue/income and expense/expenditure as raw fields on ACCOUNT — these are aggregates (sums over time), not facts about the account itself. Aggregates should be computed via queries (or materialized into a reporting table later for performance) rather than stored as columns that the app has to remember to keep in sync.
- income and total_savings — reasonable as concepts, but “income” in particular is a self-declared figure (used for credit risk / loan eligibility) rather than a derived banking fact, so it's fine to keep this as a stored field — just understand that it's qualitatively different from balance, and shouldn't be confused with money that has actually moved through the ledger.

Why this matters more in fintech than other domains:

In a social media app, a denormalized “like count” being off by one for a few seconds is invisible to anyone. In a banking app, a balance being off by one cent is a regulatory incident, a customer support ticket, and potentially a reconciliation failure that takes a human days to track down. The cost of denormalization bugs is asymmetric here — which is why fintech engineering culture leans so heavily on the ledger pattern covered in Section 7.

2.3 TRANSACTION

Original issue: cards is a list of all card ids on the transaction. In almost every real payment flow, a single transaction is funded by at most one payment instrument — one card, or a direct bank transfer with no card at all. A list here implies a transaction could be split across multiple cards simultaneously, which is a rare enough case (and complex enough to model correctly) that it shouldn't be the default shape. The fix is a single nullable card_id foreign key: present when the transaction was card-funded, null when it was an account-to-account transfer.

Missing fields a real transaction record needs: an amount (surprisingly absent from your draft — sender/receiver alone tell you who, not how much), a currency, a status (pending / settled / failed / reversed), and an idempotency_key. The idempotency key deserves its own callout, because it solves a problem every fintech engineer eventually gets paged for.

Idempotency keys — why every transaction needs one:

If a mobile app sends a ‘transfer ₦5,000’ request and the network times out before the response arrives, the app doesn't know if the transfer succeeded. The naive (and dangerous) move is to just retry — which risks double-charging if the first request actually went through. The fix used industry-wide is an idempotency_key: a unique value the client generates once per user intent. The server stores it; if the same key arrives twice, it returns the original result instead of creating a second transaction. This is precisely the kind of at-least-once-delivery problem you've already been studying for your distributed job queue — the same principle applies here.

2.4 INVESTMENTS

Original issue: `company_id` is marked as the primary key. This can't work — if 500 different users all invest in Apple stock, you'd need 500 different rows describing Apple, all sharing the same primary key, which a database will reject outright. The deeper issue is a conflation of two different things: facts about a company/security (its name, ticker, asset class) versus facts about one user's position in that security (how many shares, at what average cost).

Fix: split into two tables.

- **SECURITY** — one row per investable instrument (a stock, bond, ETF, or fund), with its own `security_id` primary key. This is reference data, shared across all users.
- **INVESTMENT_HOLDING** — one row per (account, security) pair: this user's quantity and average cost for that security. `holding_id` is its own primary key; `account_id` and `security_id` are both foreign keys.

This mirrors exactly how you'd model a library's books (**BOOK**, shared) versus who has which book checked out (**LOAN**, per-person) — a classic many-to-many resolved with a join/junction-style table, which carries its own data (quantity, cost) rather than being a bare link.

On the brokerage API sync goal: since this data will be pulled live from an external investment platform, you'll also want an **EXTERNAL_BROKERAGE_LINK** table storing which third-party account a user has connected (and an OAuth token reference, never the raw token in plaintext) plus a `last_synced_at` timestamp, so your sync job knows what's stale. This is covered fully in Section 4.8.

2.5 CARDS

Original issue: “Cards numbers (this can be a list)” is the most safety-critical mistake in the draft. Storing raw card numbers (the Primary Account Number, or PAN) is something the Payment Card Industry Data Security Standard (PCI DSS) is built specifically to prevent. The standard requires that if a PAN is stored at all, it must be rendered unreadable (via tokenization, strong encryption, or at minimum truncation), and explicitly forbids ever storing the CVV/CVC or PIN under any circumstances, even encrypted, even temporarily.

In practice, almost no fintech engineering team stores raw PANs at all — instead they integrate with a payment processor (Stripe, Paystack, Flutterwave are common in the Nigerian/African market) that handles the card capture and returns back a token. Your **CARD** table stores that token, plus a masked display version (first six digits / last four digits, per the PCI display rule), and nothing else sensitive.

The “a list” framing also has the same 1NF issue as **ACCOUNT**'s list fields: a user having multiple cards is correctly modeled as multiple rows in **CARD** (one per card, each with its own `card_id`), each pointing back to the user's `account_id` — not a list embedded in a single field.

2.6 LOANS

Original issue: `transaction_list` as an embedded list, same 1NF problem as before — solved the same way (`TRANSACTION` or `LEDGER_ENTRY` rows reference `loan_id`, rather than `LOAN` storing a list back). The more interesting gap is that “balance remaining” on a loan is itself something that should be derived, exactly like `ACCOUNT.balance`, rather than manually decremented — each repayment is a transaction/ledger event, and the remaining balance is the original principal minus the sum of all repayments applied so far.

Missing: an amortization schedule. Real loan products (and any loan feature you'd want to demo convincingly in an interview) need to know what's due and when — not just a single running balance. Section 4.6 introduces `LOAN_REPAYMENT_SCHEDULE` for this: one row per installment, with a due date, the amount expected, and whether it's been paid.

3. Foundational Principles for Fintech Data Modeling

Before looking at the revised schema, it's worth naming the principles driving every decision in it. These aren't arbitrary style preferences — each one exists because of a specific failure mode that has burned real companies, and knowing the failure mode is what makes the principle stick instead of feeling like an arbitrary rule.

3.1 Money is stored as integers, never floats

Floating-point numbers cannot represent most decimal fractions exactly in binary — $0.1 + 0.2$ famously does not equal 0.3 in IEEE-754 floating point. For most software this rounding error is invisible. For money, it compounds across millions of transactions into real, reportable discrepancies. The universal fix: store every monetary amount as an integer count of the currency's smallest unit — kobo instead of naira, cents instead of dollars — and only convert to a decimal display value at the UI layer. This is why every amount field in the revised schema is named `amount_minor` or `balance_minor`: “minor unit” is the standard term for this smallest denomination.

3.2 Double-entry accounting, not single balances

Covered in depth in Section 7, but the short version: a financial system should never let an amount move without recording two equal and opposite effects — a debit somewhere and a credit somewhere else, always summing to zero for a given transaction. This is centuries-old bookkeeping practice, not a fintech invention, and it gives you something single-balance systems can't: a mathematical self-check that catches bugs (a missing write, a duplicate write, a partial failure) automatically, rather than relying on someone noticing a customer complaint.

3.3 Normalize first, denormalize deliberately

First Normal Form (1NF) — no field holds a list or a repeating group — and Third Normal Form (3NF) — every non-key field depends only on the primary key, not on other non-key fields — are the baseline. Almost every problem in your original draft was a 1NF violation (lists embedded in fields) or a derived-value violation (storing a value, like balance, that should be computed from other rows). Once the schema is properly normalized, it's fine to add caching/denormalization later for performance — the rule is to denormalize on purpose, after measuring a real performance need, not by default out of convenience.

3.4 Immutability and append-only history for financial events

Once a ledger entry is written, it is never updated or deleted — if a transaction needs to be reversed, you write a new, opposite entry that cancels it out, leaving both the original and the reversal visible in the history. This is what makes a financial system auditable: anyone (a regulator, an internal auditor, you at 2am debugging a discrepancy) can reconstruct exactly what happened and when, because nothing was ever silently overwritten.

3.5 Idempotency at every write boundary

Any operation that moves money needs to be safe to retry. Networks fail, mobile clients time out, queue consumers occasionally redeliver a message more than once (the “at-least-once delivery” problem you’ve already studied for your job queue project). The fix is the same everywhere: every write that has a real-world side effect (charging a card, crediting an account, applying a loan repayment) is keyed by a unique idempotency token, and a retry with the same token returns the original result rather than repeating the effect.

3.6 Sensitive data minimization (PCI-DSS, and the same instinct for passwords)

The PCI Security Standards Council’s guidance is blunt: “if you don’t need it, don’t store it.” Card validation codes (CVV) and PINs may never be stored after authorization, full stop — not encrypted, not temporarily, not for any reason. Primary account numbers, if stored at all, must be tokenized or encrypted and only ever displayed masked (first six and last four digits maximum). The same minimization instinct extends to your `USER.password` field, which should hold a salted hash (bcrypt or argon2id), never the plaintext, and never even a reversible encryption of it — a hash that can’t be decrypted at all is strictly safer than an encrypted value that theoretically could be.

3.7 Reference data lives in its own table

Currency codes, card networks (Visa/Mastercard/Verve), loan types — anywhere your original draft used a free-text string for a value that’s really drawn from a small, known set, a small reference table (`CURRENCY`, `CARD_PROVIDER`) is safer. It gives you a place to hang metadata (a currency’s decimal precision, a provider’s logo URL) and a foreign key constraint that makes “typo currency codes” a database error instead of a silent data quality bug.

3.8 Compliance and audit are first-class entities, not afterthoughts

Given your prior exploration of AML/KYC concepts (CBN, NDPA, FATF-style frameworks), it’s worth calling out explicitly: a production fintech schema needs `KYC_VERIFICATION` (what identity documents were checked, when, and with what result) and an `AUDIT_LOG` (who changed what, and what it looked like before and after) as designed-in entities from day one, not bolted on after a regulator asks for them. They’re included in Section 4 at a scope appropriate for a learning project, with notes on how they’d expand in a real deployment.

4. The Revised Schema (Entity-by-Entity)

Every table below lists the field name, data type, whether it's required (NOT NULL), and a note explaining its purpose or constraint. Primary keys are marked PK, foreign keys FK. Field names use snake_case throughout for consistency with standard SQL convention (your original draft mixed cases — picking one convention and sticking to it matters more than which one you pick).

All ID fields use UUID (stored as a 36-character string, or natively as a UUID type in PostgreSQL) rather than auto-incrementing integers. This is a deliberate fintech convention: sequential integer IDs leak information (a competitor or attacker can guess how many users/transactions you have, or enumerate them), whereas UUIDs don't, and they remain unique even if you ever shard the database or merge data from multiple sources.

4.1 USER

Represents a person who has registered on the platform. Does not store account_id — see Section 2.1 for why that relationship is inverted.

Table: user

Field	Type	Required	Notes
user_id	UUID	Yes (PK)	Primary key.
username	VARCHAR(30)	Yes (Unique)	Public-facing handle.
email	VARCHAR(255)	Yes (Unique)	Used for login + notifications. Store lowercase to make uniqueness case-insensitive.
password_hash	VARCHAR(255)	Yes	bcrypt/argon2id hash. Never plaintext, never reversible encryption.
first_name	VARCHAR(80)	Yes	
last_name	VARCHAR(80)	Yes	
date_of_birth	DATE	Yes	Used for age/KYC checks.
present_address	VARCHAR(255)	Yes	Current residential address.
permanent_address	VARCHAR(255)	Yes	Corrected spelling from original draft.
city	VARCHAR(100)	Yes	
postal_code	VARCHAR(20)	No	Optional — not every country/locality uses postal codes.
country_code	CHAR(2)	Yes	ISO 3166-1 alpha-2 (e.g. NG, GB, US).
default_currency_code	CHAR(3)	Yes (FK to currency)	ISO 4217 (e.g. NGN, USD, GBP).
timezone	VARCHAR(50)	Yes	IANA timezone name, e.g. Africa/Lagos.
two_factor_enabled	BOOLEAN	Yes	Default false.
kyc_status	ENUM	Yes	unverified / pending / verified / rejected. See

Field	Type	Required	Notes
			KYC_VERIFICATION for detail history.
account_status	ENUM	Yes	active / suspended / closed.
created_at	TIMESTAMPTZ	Yes	Set once, never updated.
updated_at	TIMESTAMPTZ	Yes	Updated on every modification.

4.2 KYC_VERIFICATION

Tracks identity verification attempts. A user may need to redo this (expired ID, failed liveness check, a regulator-mandated re-verification), so this is a one-to-many child of USER rather than a flat status field alone.

Table: kyc_verification

Field	Type	Required	Notes
kyc_id	UUID	Yes (PK)	Primary key.
user_id	UUID	Yes (FK to user)	
document_type	ENUM	Yes	national_id / passport / drivers_license / bvn (Nigeria-specific).
document_reference	VARCHAR(100)	Yes	Tokenized/masked reference, not the raw document number where avoidable.
verification_status	ENUM	Yes	pending / approved / rejected.
verified_by	VARCHAR(100)	No	Internal reviewer or third-party verification provider name.
submitted_at	TIMESTAMPTZ	Yes	
reviewed_at	TIMESTAMPTZ	No	Null until a decision is made.

4.3 CURRENCY

Small reference table. Every currency-typed field elsewhere in the schema (ACCOUNT.currency_code, TRANSACTION.currency_code) is a foreign key into this table rather than a free-text string.

Table: currency

Field	Type	Required	Notes
currency_code	CHAR(3)	Yes (PK)	ISO 4217 code, e.g. NGN, USD, GBP.
display_name	VARCHAR(50)	Yes	e.g. "Nigerian Naira".
symbol	VARCHAR(5)	Yes	e.g. currency symbol character(s).
minor_unit_decimals	SMALLINT	Yes	Number of decimal places in the minor unit — 2 for NGN/USD/GBP, 0 for JPY.

4.4 ACCOUNT

Represents one financial account belonging to one user (a user may have several — current, savings, investment wallet). Notice what's absent compared to the original draft: no transaction list, no card list, no debit/credit lists, no invoice list, and balance is described as derived rather than a freely-writable column.

Table: account

Field	Type	Required	Notes
account_id	UUID	Yes (PK)	Primary key.
user_id	UUID	Yes (FK to user)	Owning user. One user, many accounts.
account_type	ENUM	Yes	current / savings / investment_wallet / loan_holding.
account_number	VARCHAR(20)	Yes (Unique)	Customer-facing account number (e.g. NUBAN in Nigeria).
currency_code	CHAR(3)	Yes (FK to currency)	The account's operating currency.
balance_minor	BIGINT	Yes	Cached/derived from LEDGER_ENTRY — see Section 7.1. Never written to directly by application code; only by the ledger-posting routine.
income_minor	BIGINT	No	Self-declared income, in minor units. Used for credit/loan risk scoring — not a banking fact, just declared data.
total_savings_minor	BIGINT	No	Optional rollup target the user is saving toward; distinct from balance.
account_status	ENUM	Yes	active / frozen / closed.
created_at	TIMESTAMPTZ	Yes	
updated_at	TIMESTAMPTZ	Yes	

On removing the embedded lists:

Anywhere the original draft had “a list of X ids,” the fix is always the same: the X table already has a foreign key pointing back to account_id. That foreign key is the relationship. To get “all cards for this account,” you query for cards where account_id matches — you never store the reverse list. This single idea removes transaction, Debit_Transaction, Credit_Transaction, Cards, and invoices from ACCOUNT entirely, and the schema is no less expressive for it — it's more correct, more queryable, and impossible to get out of sync.

4.5 LEDGER_ENTRY

This is the new table that does not exist in the original draft and is the architectural core of the whole revision. Every movement of money — a transfer, a card payment, a loan disbursement,

a fee — produces rows here, never just an update to ACCOUNT.balance directly. Full mechanics and a worked example are in Section 7.1; this is the field-level reference.

Table: ledger_entry

Field	Type	Required	Notes
entry_id	UUID	Yes (PK)	Primary key.
account_id	UUID	Yes (FK to account)	Which account this movement affects.
transaction_id	UUID	Yes (FK to transaction)	The business event this entry belongs to.
direction	ENUM	Yes	DEBIT or CREDIT. Exactly two entries per transaction_id, one of each, with matching amount_minor.
amount_minor	BIGINT	Yes	Always positive; direction conveys sign, never a negative number.
currency_code	CHAR(3)	Yes (FK to currency)	Should match the account's currency; cross-currency transfers go through an FX conversion step, not a mismatched ledger row.
running_balance_after_minor	BIGINT	Yes	Account balance immediately after this entry — makes balance history queryable without replaying the whole ledger.
created_at	TIMESTAMPTZ	Yes	Append-only: this table has no updated_at, because rows are never updated.

This table is append-only by design:

There is deliberately no UPDATE or DELETE path for ledger_entry in application code — only INSERT. If a payment needs to be reversed, you insert a new pair of entries that cancel the original out (a debit where there was a credit, and vice versa), rather than touching the original rows. This is what makes the table auditable: the full history of what happened, and when, is always intact.

4.6 TRANSACTION

Represents the business-level event — “Seyi sent Feranmi 5,000” — as distinct from its accounting effect (which lives in LEDGER_ENTRY). Keeping these separate matters because a single business transaction sometimes has more than a simple 1:1 mapping to ledger postings once you add fees: a 5,000 transfer with a 50 platform fee is one TRANSACTION but three LEDGER_ENTRY rows (debit sender 5,050, credit receiver 5,000, credit platform-revenue-account 50).

Table: transaction

Field	Type	Required	Notes
transaction_id	UUID	Yes (PK)	Primary key.
idempotency_key	VARCHAR(100)	Yes	Client-generated. A retry with the same key

Field	Type	Required	Notes
		(Unique)	returns the original transaction instead of creating a duplicate. See Section 7.2.
transaction_type	ENUM	Yes	transfer / card_payment / loan_disbursement / loan_repayment / fee / refund.
sender_account_id	UUID	No (FK to account)	Null for inbound-only events like a loan disbursement.
receiver_account_id	UUID	No (FK to account)	Null for outbound-only events like a withdrawal to an external bank.
card_id	UUID	No (FK to card)	Set only when the transaction was funded by a card; null for direct account transfers.
loan_id	UUID	No (FK to loan)	Set only for loan disbursement/repayment transactions.
amount_minor	BIGINT	Yes	The principal amount of the transaction, excluding fees.
currency_code	CHAR(3)	Yes (FK to currency)	
status	ENUM	Yes	pending / settled / failed / reversed.
description	VARCHAR(255)	No	User-facing memo/narration.
created_at	TIMESTAMPTZ	Yes	
settled_at	TIMESTAMPTZ	No	Null until status moves to settled.

Note that `sender_account_id` and `receiver_account_id` are two separate foreign keys pointing at the same table (ACCOUNT) — this is a normal and common pattern called a “role-based” or “recursive” foreign key relationship; the same parent table is referenced twice with different meanings each time.

4.7 CARD_PROVIDER

Small reference table for the card network (or in-house issuing partner). Keeps TRANSACTION/CARD from storing free-text provider names.

Table: card_provider

Field	Type	Required	Notes
provider_id	UUID	Yes (PK)	
provider_name	VARCHAR(50)	Yes	e.g. Visa, Mastercard, Verve.
country_scope	VARCHAR(100)	No	Optional — some local schemes (e.g. Verve) are primarily regional.

4.8 CARD

This table is the most safety-sensitive in the whole schema. It must never contain a raw, usable card number, and must never contain a CVV or PIN under any circumstances — see Section

2.5 and Section 3.6. `card_token` is the value returned by your payment processor's tokenization vault (Paystack, Flutterwave, Stripe, etc.); your application uses the token to initiate charges, and the processor (not your database) holds the reversible mapping back to the real PAN.

Table: card

Field	Type	Required	Notes
<code>card_id</code>	UUID	Yes (PK)	Primary key.
<code>account_id</code>	UUID	Yes (FK to account)	Which account this card draws from / is linked to.
<code>provider_id</code>	UUID	Yes (FK to card_provider)	
<code>card_token</code>	VARCHAR(255)	Yes (Unique)	Opaque token from the payment processor's vault. This is what the app actually uses to charge the card — never a raw PAN.
<code>pan_bin</code>	CHAR(6)	Yes	First 6 digits (Bank Identification Number) — safe to store, used for routing/issuer display.
<code>pan_last_four</code>	CHAR(4)	Yes	Last 4 digits — safe to store, used for customer-facing display (masked card number).
<code>cardholder_name</code>	VARCHAR(100)	Yes	
<code>expiry_month</code>	SMALLINT	Yes	
<code>expiry_year</code>	SMALLINT	Yes	
<code>card_status</code>	ENUM	Yes	active / expired / blocked / replaced.
<code>created_at</code>	TIMESTAMPTZ	Yes	

Never stored, under any circumstances:

The full Primary Account Number (PAN) in plaintext, the CVV/CVC security code, and the PIN or PIN block. PCI DSS prohibits storing CVV/PIN after authorization even temporarily and even encrypted — there is no compliant way to keep them. If a feature ever seems to need the CVV after the initial charge (it doesn't, normally), that's a sign the architecture needs a payment processor doing the heavy lifting, not a new database column.

4.9 LOAN

Represents a loan extended to an account. As with `ACCOUNT.balance`, `balance_remaining_minor` is conceptually derived (principal minus the sum of settled repayment transactions) but typically cached on the row for fast reads, with the ledger as the source of truth if it's ever in doubt.

Table: loan

Field	Type	Required	Notes
<code>loan_id</code>	UUID	Yes (PK)	Primary key.
<code>account_id</code>	UUID	Yes (FK to	The account the loan was disbursed into and is

Field	Type	Required	Notes
		account)	repaid from.
loan_type	ENUM	Yes	personal / mortgage / cash_advance / asset_finance.
principal_minor	BIGINT	Yes	The original amount disbursed.
interest_rate_bps	INTEGER	Yes	Interest rate in basis points (1/100th of a percent) -- avoids float rounding issues, same reasoning as Section 3.1.
currency_code	CHAR(3)	Yes (FK to currency)	
balance_remaining_minor	BIGINT	Yes	Cached; derivable from principal minus settled repayments.
loan_status	ENUM	Yes	pending_approval / active / repaid / defaulted.
disbursed_at	TIMESTAMPTZ	No	Null until approved and paid out.
created_at	TIMESTAMPTZ	Yes	

Note there is no transaction_list field. Every disbursement and repayment is a row in TRANSACTION with loan_id set, and the full history for a loan is queryable as all transactions where loan_id matches this loan -- exactly the same pattern used everywhere else in this schema.

4.10 LOAN_REPAYMENT_SCHEDULE

New table, not present in the original draft. Without this, a loan has a single running balance but no way to express what is due, and when -- which is the feature users (and interviewers, if this comes up in a system design discussion) actually expect from a loan product.

Table: loan_repayment_schedule

Field	Type	Required	Notes
schedule_id	UUID	Yes (PK)	Primary key.
loan_id	UUID	Yes (FK to loan)	
installment_number	SMALLINT	Yes	1, 2, 3... in sequence.
due_date	DATE	Yes	
installment_amount_minor	BIGINT	Yes	Principal + interest portion due this installment.
paid_flag	BOOLEAN	Yes	Default false; set true once a matching repayment transaction settles.
paid_transaction_id	UUID	No (FK to transaction)	Set once paid -- links the schedule row to the actual repayment event.

4.11 INVOICE

Represents an amount owed by or to an account -- useful for bill payments, merchant invoicing, or subscription-style charges, tying back to a settling transaction once paid.

Table: invoice

Field	Type	Required	Notes
invoice_id	UUID	Yes (PK)	Primary key.
account_id	UUID	Yes (FK to account)	The account this invoice is billed to.
settling_transaction_id	UUID	No (FK to transaction)	Null until paid; set to the transaction that settled it.
amount_due_min_or	BIGINT	Yes	
currency_code	CHAR(3)	Yes (FK to currency)	
due_date	DATE	Yes	
invoice_status	ENUM	Yes	open / paid / overdue / cancelled.
created_at	TIMESTAMPTZ	Yes	

4.12 SECURITY

Reference data describing one investable instrument -- shared across every user who holds it. See Section 2.4 for why this had to be split out from the original single INVESTMENTS table.

Table: security

Field	Type	Required	Notes
security_id	UUID	Yes (PK)	Primary key.
ticker_symbol	VARCHAR(15)	Yes	e.g. AAPL, DANGCEM.
company_name	VARCHAR(150)	Yes	
asset_type	ENUM	Yes	stock / bond / etf / mutual_fund.
exchange	VARCHAR(50)	No	e.g. NASDAQ, NGX (Nigerian Exchange).
currency_code	CHAR(3)	Yes (FK to currency)	The currency the security is priced in.

4.13 INVESTMENT_HOLDING

One row per (account, security) pair -- this user's actual position. This is the table that answers what does this user own, which the original INVESTMENTS table couldn't represent correctly once its primary key was set to company_id.

Table: investment_holding

Field	Type	Required	Notes
holding_id	UUID	Yes (PK)	Primary key.
account_id	UUID	Yes (FK to account)	The investment wallet account this position sits in.
security_id	UUID	Yes (FK to security)	
external_link_id	UUID	No (FK to external_brokerage_link)	Set when this holding's data is synced from a connected external brokerage; null for positions bought directly through this platform.
quantity	DECIMAL(18,6)	Yes	Shares/units held. Decimal because fractional shares are common.
average_cost_minor	BIGINT	Yes	Average price paid per unit, in minor currency units.
last_synced_at	TIMESTAMPTZ	No	Null for directly-purchased holdings; set on each successful sync for externally-linked ones.

4.14 EXTERNAL_BROKERAGE_LINK

Supports the goal of pulling investment data live from a third-party platform via API. Stores the connection itself -- never a raw, long-lived secret in plaintext.

Table: external_brokerage_link

Field	Type	Required	Notes
link_id	UUID	Yes (PK)	Primary key.
user_id	UUID	Yes (FK to user)	
provider_name	VARCHAR(50)	Yes	e.g. the brokerage/investment platform's name.
oauth_token_reference	VARCHAR(255)	Yes	Reference/handle into a secrets vault holding the real OAuth token -- not the token itself in this table.
link_status	ENUM	Yes	active / expired / revoked.
last_synced_at	TIMESTAMPTZ	No	Drives the sync job's polling/refresh logic.
created_at	TIMESTAMPTZ	Yes	

On the brokerage sync architecture:

A background job (a strong fit for the same job-queue pattern you're already building) periodically reads each active external_brokerage_link, calls the third-party API using the vault-stored token, and upserts rows into investment_holding. Keeping the sync logic in a queued job rather than a synchronous request means a slow or rate-limited third-party API never blocks your own app's response times -- the same autoscaling/concurrency reasoning you've been applying to your distributed job queue project applies directly here.

4.15 AUDIT_LOG

Records who changed what, and what the data looked like before and after. New table, included because a fintech system without this is effectively unauditible -- which matters both for regulatory reasons and for your own debugging sanity.

Table: audit_log

Field	Type	Required	Notes
audit_id	UUID	Yes (PK)	Primary key.
actor_id	UUID	No	The user or system process that performed the action; null for automated system jobs without a user context.
entity_type	VARCHAR(50)	Yes	e.g. account, card, loan -- which table was affected.
entity_id	UUID	Yes	The primary key of the affected row.
action	ENUM	Yes	create / update / delete / status_change.
before_state	JSONB	No	Snapshot of the row before the change.
after_state	JSONB	No	Snapshot of the row after the change.
created_at	TIMESTAMPTZ	Yes	

5. Relationship Map and ER Diagrams

This section gives the complete picture in two layers: first a table of every relationship in the schema with its cardinality (one-to-many, many-to-many, etc.) stated explicitly, then the visual ER diagrams themselves.

5.1 Full relationship table

Relationships

Relationship	Cardinality	Meaning
USER -> ACCOUNT	1 : N	A user can hold several accounts (current, savings, investment wallet).
USER -> KYC_VERIFICATION	1 : N	A user may submit identity verification more than once over time.
USER -> EXTERNAL_BROKERAGE_LINK	1 : N	A user may connect more than one external investment platform.
ACCOUNT -> LEDGER_ENTRY	1 : N	Every balance movement on an account is one ledger row.
ACCOUNT -> TRANSACTION (sender)	1 : N	An account can be the sending side of many transactions.
ACCOUNT -> TRANSACTION (receiver)	1 : N	An account can be the receiving side of many transactions.
TRANSACTION -> LEDGER_ENTRY	1 : 2 (exactly)	Double-entry rule: one debit row and one credit row per transaction (more rows if fees are involved -- see Section 4.6).
ACCOUNT -> CARD	1 : N	An account can have several linked cards.
CARD_PROVIDER -> CARD	1 : N	One network (Visa, Mastercard, Verve) issues many cards platform-wide.
TRANSACTION -> CARD	N : 1 (optional)	Many transactions can be funded by the same card; a transaction has at most one card, or none.
ACCOUNT -> LOAN	1 : N	An account can have multiple loans over time.
LOAN -> LOAN_REPAYMENT_SCHEDULE	1 : N	One row per installment due-date.
LOAN -> TRANSACTION	1 : N	Disbursement and every repayment are individual transaction rows referencing the loan.
ACCOUNT -> INVOICE	1 : N	An account can have several invoices, paid or unpaid.
TRANSACTION -> INVOICE	1 : 1 (optional)	An invoice is settled by at most one transaction.
ACCOUNT -> INVESTMENT_HOLDING	1 : N	An investment-wallet account can hold many positions.
SECURITY -> INVESTMENT_HOLDING	1 : N	One security (e.g. a single stock) can be held by many different accounts.
EXTERNAL_BROKERAGE_LINK -	1 : N	One connected brokerage link can supply many synced

Relationship	Cardinality	Meaning
> INVESTMENT_HOLDING		holdings.
(any entity) -> AUDIT_LOG	1 : N (conceptual)	Every mutable entity in the system can generate audit rows; modeled generically via entity_type + entity_id rather than a literal foreign key to every table.

5.2 High-level entity relationship diagram

This diagram shows all entities and how they connect. Dashed lines indicate a relationship that exists but is slightly less direct (for example, CARD relates back to ACCOUNT, but the diagram keeps the primary flow -- USER through ACCOUNT through LEDGER_ENTRY through TRANSACTION -- as the solid, central path, since that's the spine of the whole system).

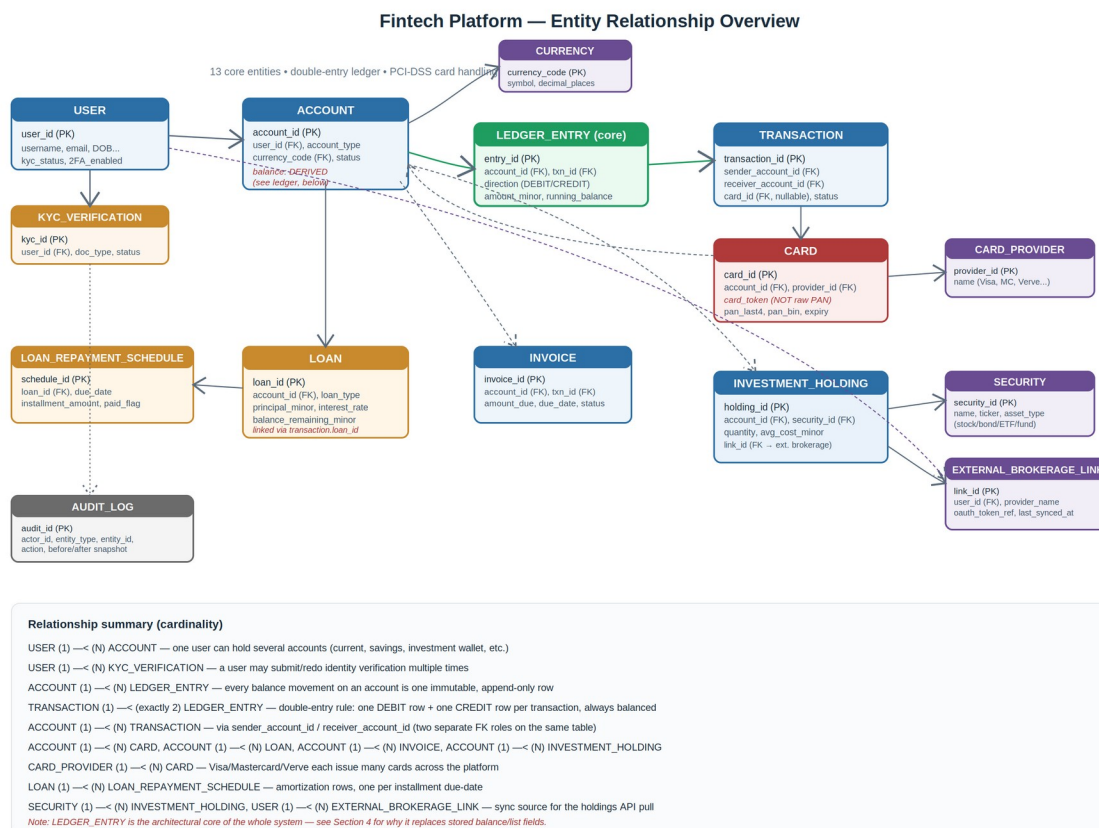
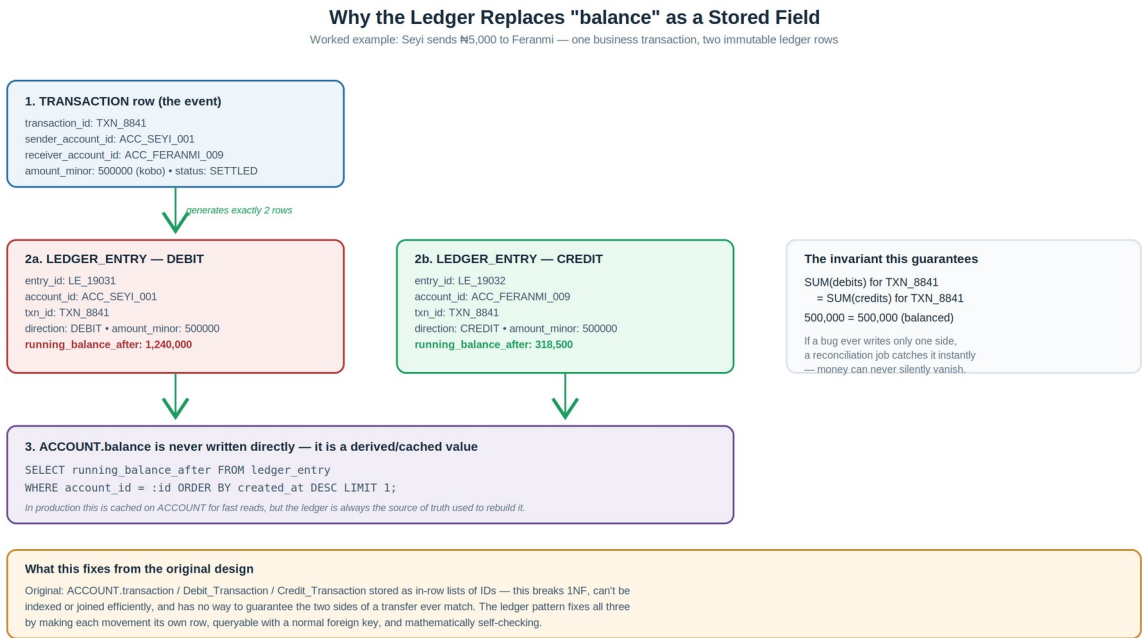


Figure 1. Full entity relationship overview -- 13 core entities.

5.3 Double-entry ledger mechanics, worked example

Because LEDGER_ENTRY is the single most important new idea in this revision, it gets its own diagram showing exactly what happens, end to end, for one transfer -- from the business-level TRANSACTION row, to the two LEDGER_ENTRY rows it generates, to how ACCOUNT.balance is ultimately derived from them.



6. SQL DDL Reference Implementation

Below is a working PostgreSQL DDL script implementing the full schema. PostgreSQL is the right choice to actually build this against -- it has native UUID and JSONB support, strong constraint enforcement, and is the database most fintech-style system design interviews assume by default. Run this against a fresh database to get a real, queryable schema rather than just a diagram.

6.1 Extensions and enum types

```
-- Enable UUID generation
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Enum types used across multiple tables
CREATE TYPE account_type_enum AS ENUM
  ('current', 'savings', 'investment_wallet', 'loan_holding');
CREATE TYPE account_status_enum AS ENUM ('active', 'frozen', 'closed');
CREATE TYPE kyc_status_enum AS ENUM
  ('unverified', 'pending', 'verified', 'rejected');
CREATE TYPE ledger_direction_enum AS ENUM ('DEBIT', 'CREDIT');
CREATE TYPE transaction_type_enum AS ENUM
  ('transfer', 'card_payment', 'loan_disbursement',
   'loan_repayment', 'fee', 'refund');
CREATE TYPE transaction_status_enum AS ENUM
  ('pending', 'settled', 'failed', 'reversed');
CREATE TYPE card_status_enum AS ENUM
  ('active', 'expired', 'blocked', 'replaced');
CREATE TYPE loan_type_enum AS ENUM
  ('personal', 'mortgage', 'cash_advance', 'asset_finance');
CREATE TYPE loan_status_enum AS ENUM
  ('pending_approval', 'active', 'repaid', 'defaulted');
CREATE TYPE invoice_status_enum AS ENUM
  ('open', 'paid', 'overdue', 'cancelled');
CREATE TYPE asset_type_enum AS ENUM
  ('stock', 'bond', 'etf', 'mutual_fund');
```

6.2 Reference tables

```
CREATE TABLE currency (
  currency_code      CHAR(3) PRIMARY KEY,
  display_name       VARCHAR(50) NOT NULL,
  symbol             VARCHAR(5) NOT NULL,
  minor_unit_decimals SMALLINT NOT NULL DEFAULT 2
);

CREATE TABLE card_provider (
  provider_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  provider_name       VARCHAR(50) NOT NULL,
  country_scope       VARCHAR(100)
);
```

6.3 USER and KYC_VERIFICATION

```
CREATE TABLE app_user (
  user_id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username            VARCHAR(30) NOT NULL UNIQUE,
  email              VARCHAR(255) NOT NULL UNIQUE,
```

```

password_hash      VARCHAR(255) NOT NULL,
first_name         VARCHAR(80)  NOT NULL,
last_name          VARCHAR(80)  NOT NULL,
date_of_birth      DATE NOT NULL,
present_address    VARCHAR(255) NOT NULL,
permanent_address  VARCHAR(255) NOT NULL,
city               VARCHAR(100) NOT NULL,
postal_code        VARCHAR(20),
country_code       CHAR(2) NOT NULL,
default_currency_code CHAR(3) NOT NULL REFERENCES currency(currency_code),
timezone           VARCHAR(50) NOT NULL,
two_factor_enabled BOOLEAN NOT NULL DEFAULT FALSE,
kyc_status          kyc_status_enum NOT NULL DEFAULT 'unverified',
account_status     account_status_enum NOT NULL DEFAULT 'active',
created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- NOTE: table named app_user, not user, because USER is a reserved
-- keyword in PostgreSQL.

CREATE TABLE kyc_verification (
  kyc_id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES app_user(user_id),
  document_type     VARCHAR(30) NOT NULL,
  document_reference VARCHAR(100) NOT NULL,
  verification_status VARCHAR(20) NOT NULL DEFAULT 'pending',
  verified_by       VARCHAR(100),
  submitted_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  reviewed_at       TIMESTAMPTZ
);
CREATE INDEX idx_kyc_user ON kyc_verification(user_id);

```

6.4 ACCOUNT

```

CREATE TABLE account (
  account_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES app_user(user_id),
  account_type      account_type_enum NOT NULL,
  account_number    VARCHAR(20) NOT NULL UNIQUE,
  currency_code     CHAR(3) NOT NULL REFERENCES currency(currency_code),
  balance_minor     BIGINT NOT NULL DEFAULT 0,
  income_minor      BIGINT,
  total_savings_minor BIGINT,
  account_status    account_status_enum NOT NULL DEFAULT 'active',
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT balance_non_negative CHECK (balance_minor >= 0)
);
CREATE INDEX idx_account_user ON account(user_id);

```

6.5 TRANSACTION and LEDGER_ENTRY (the core of the system)

```

CREATE TABLE transaction (
  transaction_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  idempotency_key   VARCHAR(100) NOT NULL UNIQUE,
  transaction_type  transaction_type_enum NOT NULL,
  sender_account_id UUID REFERENCES account(account_id),
  receiver_account_id UUID REFERENCES account(account_id),
  card_id           UUID, -- FK added after card table exists
  loan_id           UUID, -- FK added after loan table exists
  amount_minor      BIGINT NOT NULL CHECK (amount_minor > 0),

```

```

    currency_code      CHAR(3) NOT NULL REFERENCES currency(currency_code),
    status              transaction_status_enum NOT NULL DEFAULT 'pending',
    description         VARCHAR(255),
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    settled_at          TIMESTAMPTZ
);
CREATE INDEX idx_txn_sender    ON transaction(sender_account_id);
CREATE INDEX idx_txn_receiver ON transaction(receiver_account_id);

CREATE TABLE ledger_entry (
    entry_id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_id          UUID NOT NULL REFERENCES account(account_id),
    transaction_id       UUID NOT NULL REFERENCES transaction(transaction_id),
    direction           ledger_direction_enum NOT NULL,
    amount_minor        BIGINT NOT NULL CHECK (amount_minor > 0),
    currency_code        CHAR(3) NOT NULL REFERENCES currency(currency_code),
    running_balance_after_minor BIGINT NOT NULL,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_ledger_account ON ledger_entry(account_id, created_at);
CREATE INDEX idx_ledger_txn     ON ledger_entry(transaction_id);

-- This partial unique-ish constraint is the closest a single CHECK
-- can get to enforcing 'exactly one debit and one credit per
-- transaction' -- the full guarantee is enforced in the application's
-- ledger-posting routine (see Section 7.1), which always writes both
-- rows in a single DB transaction (BEGIN ... COMMIT), never one
-- without the other.

```

6.6 CARD

```

CREATE TABLE card (
    card_id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_id         UUID NOT NULL REFERENCES account(account_id),
    provider_id        UUID NOT NULL REFERENCES card_provider(provider_id),
    card_token         VARCHAR(255) NOT NULL UNIQUE,
    pan_bin            CHAR(6) NOT NULL,
    pan_last_four      CHAR(4) NOT NULL,
    cardholder_name    VARCHAR(100) NOT NULL,
    expiry_month       SMALLINT NOT NULL CHECK (expiry_month BETWEEN 1 AND 12),
    expiry_year        SMALLINT NOT NULL,
    card_status        card_status_enum NOT NULL DEFAULT 'active',
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_card_account ON card(account_id);

ALTER TABLE transaction
    ADD CONSTRAINT fk_txn_card FOREIGN KEY (card_id) REFERENCES card(card_id);

```

6.7 LOAN and LOAN_REPAYMENT_SCHEDULE

```

CREATE TABLE loan (
    loan_id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    account_id         UUID NOT NULL REFERENCES account(account_id),
    loan_type          loan_type_enum NOT NULL,
    principal_minor    BIGINT NOT NULL CHECK (principal_minor > 0),
    interest_rate_bps  INTEGER NOT NULL,
    currency_code      CHAR(3) NOT NULL REFERENCES currency(currency_code),
    balance_remaining_minor BIGINT NOT NULL,
    loan_status        loan_status_enum NOT NULL DEFAULT 'pending_approval',
    disbursed_at       TIMESTAMPTZ,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

);
CREATE INDEX idx_loan_account ON loan(account_id);

ALTER TABLE transaction
  ADD CONSTRAINT fk_txn_loan FOREIGN KEY (loan_id) REFERENCES loan(loan_id);

CREATE TABLE loan_repayment_schedule (
  schedule_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  loan_id              UUID NOT NULL REFERENCES loan(loan_id),
  installment_number    SMALLINT NOT NULL,
  due_date             DATE NOT NULL,
  installment_amount_minor BIGINT NOT NULL,
  paid_flag            BOOLEAN NOT NULL DEFAULT FALSE,
  paid_transaction_id   UUID REFERENCES transaction(transaction_id)
);
CREATE INDEX idx_schedule_loan ON loan_repayment_schedule(loan_id);

```

6.8 INVOICE, SECURITY, INVESTMENT_HOLDING, EXTERNAL_BROKERAGE_LINK

```

CREATE TABLE invoice (
  invoice_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account_id          UUID NOT NULL REFERENCES account(account_id),
  settling_transaction_id UUID REFERENCES transaction(transaction_id),
  amount_due_minor    BIGINT NOT NULL,
  currency_code        CHAR(3) NOT NULL REFERENCES currency(currency_code),
  due_date            DATE NOT NULL,
  invoice_status       invoice_status_enum NOT NULL DEFAULT 'open',
  created_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE security (
  security_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ticker_symbol        VARCHAR(15) NOT NULL,
  company_name         VARCHAR(150) NOT NULL,
  asset_type           asset_type_enum NOT NULL,
  exchange             VARCHAR(50),
  currency_code        CHAR(3) NOT NULL REFERENCES currency(currency_code)
);

CREATE TABLE external_brokerage_link (
  link_id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id             UUID NOT NULL REFERENCES app_user(user_id),
  provider_name        VARCHAR(50) NOT NULL,
  oauth_token_reference VARCHAR(255) NOT NULL,
  link_status          VARCHAR(20) NOT NULL DEFAULT 'active',
  last_synced_at      TIMESTAMPTZ,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE investment_holding (
  holding_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account_id          UUID NOT NULL REFERENCES account(account_id),
  security_id          UUID NOT NULL REFERENCES security(security_id),
  external_link_id     UUID REFERENCES external_brokerage_link(link_id),
  quantity            DECIMAL(18,6) NOT NULL,
  average_cost_minor  BIGINT NOT NULL,
  last_synced_at      TIMESTAMPTZ,
  UNIQUE(account_id, security_id)
);

```

6.9 AUDIT_LOG

```
CREATE TABLE audit_log (  
  audit_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  actor_id      UUID,  
  entity_type   VARCHAR(50) NOT NULL,  
  entity_id     UUID NOT NULL,  
  action        VARCHAR(20) NOT NULL,  
  before_state  JSONB,  
  after_state   JSONB,  
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX idx_audit_entity ON audit_log(entity_type, entity_id);
```

7. Architectural Patterns Worth Understanding Deeply

This section is written specifically for interview preparation, since you mentioned wanting to understand the reasoning behind design decisions rather than memorize the shapes. Each pattern below is something an interviewer asking a payments/fintech system design question will expect you to bring up unprompted, and to be able to justify under follow-up questions.

7.1 The ledger-posting transaction, step by step

When your application processes a transfer of 5,000 from one user to another, the actual sequence of database operations should be:

1. Check the idempotency_key has not been seen before (Section 7.2). If it has, return the existing result immediately and stop.
2. Open a single database transaction (BEGIN).
3. Insert one row into TRANSACTION with status = pending.
4. Insert a DEBIT ledger_entry for the sender's account, and a CREDIT ledger_entry for the receiver's account -- same transaction_id, equal amount_minor.
5. Update both accounts' cached balance_minor (or rely on a trigger / application-layer recompute) so reads stay fast.
6. Update the TRANSACTION row's status to settled.
7. Commit (COMMIT). If any step fails, the whole transaction rolls back -- you never end up with one ledger row and not its pair.

The phrase to know for an interview: this is sometimes called the saga or unit-of-work pattern when it spans multiple services rather than one database transaction (for example, if the card charge happens via an external payment processor's API call, which cannot be rolled back by a database ROLLBACK). In that case you need a different mechanism -- typically an outbox pattern, where the local DB transaction also writes an event to an outbox table, and a separate worker reliably delivers that event to the external system, retrying safely because of idempotency.

7.2 Idempotency keys in practice

An idempotency key is generated once by the client at the moment the user expresses intent (taps Send), and is sent with every retry of that same request. The server's job:

- On first sight of a key: process the request normally, store the key alongside the resulting transaction_id and a snapshot of the response.
- On a repeated key: skip processing entirely, return the stored response. The user (or their app, retrying automatically) gets the same answer whether this is the first attempt or the fifth.

- Keys are typically given a retention window (24-48 hours is common) after which they can be purged -- you do not need to remember an idempotency key forever, just long enough to cover realistic retry windows.

This is the same at-least-once-delivery problem you have been working through for your Redis-backed job queue: a worker can crash after doing the work but before acknowledging the message, causing the message to be redelivered. Idempotency keys are the application-level fix that makes redelivery safe, complementing rather than replacing whatever delivery guarantees the queue itself provides.

7.3 Why balance is cached, not purely computed on every read

Strictly, `balance_minor` does not need to be a stored column at all -- it could always be calculated as the sum of all `ledger_entry` rows for the account, debits negative, credits positive. For an account with millions of historical entries, running that aggregation on every single balance check (which happens on practically every screen of a banking app) would be far too slow.

The practical answer used industry-wide: cache the balance on the `ACCOUNT` row, updated atomically inside the same database transaction that writes the ledger entries (Section 7.1, step 5), and treat the ledger as the disaster-recovery source of truth -- if the cached value is ever in doubt, a reconciliation job recomputes it from the full ledger history and compares. This is a textbook example of denormalizing deliberately, after identifying a real performance need, as described in Section 3.3.

7.4 Concurrency control: why two people cannot double-spend the same balance

Picture two concurrent withdrawal requests against an account with 1,000 in it, each trying to withdraw 800. Without protection, both could read a balance of 1,000, both decide there is enough, and both succeed -- leaving the account at negative 600, which should be impossible. Two standard fixes:

- Row-level locking (`SELECT ... FOR UPDATE`): the first transaction to touch the account row locks it; the second has to wait until the first commits or rolls back, at which point it re-reads the now-correct balance.
- Optimistic locking with a version number: read the balance and a version column together; when writing, include a `WHERE` clause requiring the version to still match what you read -- if another transaction already changed it, your write affects zero rows, and your application retries with a fresh read.

This connects directly to the load-testing and concurrency-model material you have been studying -- a payments API is one of the clearest real-world cases where getting concurrency control wrong has financial, not just cosmetic, consequences.

7.5 Money as integers, restated with the bug it prevents

If amount were stored as a float and you summed one million transactions of a fractional unit each, the floating-point rounding error compounds into a balance that is measurably wrong -- not by a dramatic amount, but by an amount a reconciliation process will flag, and that you will not be able to explain by reading the code, because the bug is in how the number itself is represented, not in any visible logic error. Storing amounts as integers (BIGINT, in minor units) sidesteps the entire class of bug. This is one of the highest-leverage, lowest-effort decisions in the whole schema.

7.6 Reconciliation as a standing process, not a one-time check

Real financial systems run scheduled reconciliation jobs that re-verify the ledger's internal invariant (every transaction's debits equal its credits) and cross-check the cached `ACCOUNT.balance` against a fresh sum of that account's ledger entries. This is exactly the kind of recurring background job your Redis-backed job queue project is built to run -- it is worth explicitly building a reconcile-all-accounts job into that project as a demonstration, since it ties your two stated goals, the job queue project and fintech system design fluency, into one piece of work.

8. Suggested Build Roadmap

If the goal is to actually implement this (rather than only study it for interviews), building it in the order below means each stage produces something runnable and testable before you move to the next, and mirrors how a real engineering team would sequence the work.

Stage 1 - Core identity and accounts

- USER, KYC_VERIFICATION, CURRENCY, ACCOUNT.
- Goal: register a user, create an account, see a balance of zero. No money movement yet.

Stage 2 - The ledger and basic transfers

- TRANSACTION, LEDGER_ENTRY.
- Implement the ledger-posting routine from Section 7.1 exactly, including the idempotency check, inside one real database transaction.
- Goal: transfer money between two accounts and prove, by querying ledger_entry directly, that debits equal credits for every transaction.
- This is the stage to write your first automated test that intentionally tries to break the system: fire the same request twice with the same idempotency key and assert only one transaction was created; fire two concurrent withdrawals against a low balance and assert the account never goes negative.

Stage 3 - Cards

- CARD_PROVIDER, CARD.
- Integrate a real tokenizing payment processor sandbox (Paystack and Flutterwave both offer test-mode sandboxes well suited to a Nigeria-based project) rather than simulating tokenization yourself -- this matches your stated preference for building real, non-simulated implementations.
- Goal: a card-funded TRANSACTION row with a populated card_id, and a CARD table that never once contains a usable raw card number.

Stage 4 - Loans

- LOAN, LOAN_REPAYMENT_SCHEDULE.
- Goal: disburse a loan (a TRANSACTION crediting the user's account), generate an amortization schedule, and process repayments that mark schedule rows paid and reduce balance_remaining_minor.

Stage 5 - Invoices

- INVOICE, linked to a settling TRANSACTION.

Stage 6 - Investments and the brokerage sync

- SECURITY, INVESTMENT_HOLDING, EXTERNAL_BROKERAGE_LINK.
- This is the natural integration point with your distributed job queue project: build the brokerage-sync job as a queued, retryable task with backoff, exactly like the worker patterns you have already been implementing, rather than a synchronous endpoint.

Stage 7 - Compliance and observability

- AUDIT_LOG, plus the reconciliation job from Section 7.6.
- Goal: deliberately corrupt one account's cached balance in a test environment, run the reconciliation job, and confirm it detects and reports the discrepancy.

A note on what to emphasize in an interview write-up of this project

If you use this build as supporting material in interviews, the strongest things to lead with are not the entity list (interviewers have seen a hundred CRUD schemas) but the three decisions that show systems thinking: the double-entry ledger replacing a naive balance column, the idempotency strategy for safe retries, and the concurrency control approach that prevents double-spending. Those three, explained with the reasoning in Section 7 rather than just named, are what separates a schema you copied from one you actually understand -- which is precisely the distinction you said you are optimizing for.

9. Glossary and Further Reading

9.1 Glossary

Key terms used in this document

Term	Meaning
1NF (First Normal Form)	A normalization rule stating every field holds a single atomic value, never a list or repeating group.
3NF (Third Normal Form)	A normalization rule stating every non-key field depends only on the table's primary key, not on other non-key fields.
Double-entry accounting	A bookkeeping method where every financial movement is recorded as two equal and opposite entries, a debit and a credit, which must always balance to zero.
Idempotency key	A unique value generated once per client intent, used so that retried requests produce the same result instead of repeating a side effect like a duplicate charge.
Ledger	An append-only, immutable record of every individual financial movement, used as the authoritative source of truth for balances.
Minor unit	The smallest denomination of a currency, kobo for naira or cents for dollars, used as the unit for storing all monetary amounts as integers.
PAN (Primary Account Number)	The full card number printed on a payment card, the specific data PCI DSS governs the storage of.
PCI DSS	Payment Card Industry Data Security Standard, the security standard governing how payment card data must be protected, stored, and displayed.
Tokenization	Replacing sensitive data such as a card's PAN with a non-sensitive placeholder token that has no exploitable value if stolen, with the real value held only by a secured vault or processor.
UUID	Universally Unique Identifier, a 128-bit identifier used as a primary key in place of sequential integers, chosen so IDs do not leak information like total row counts.
Outbox pattern	A reliability pattern where a local database transaction also writes an event to an outbox table, which a separate worker reliably delivers to an external system, so the local write and the external effect cannot become inconsistent.
Saga pattern	A way of coordinating a sequence of operations across multiple services or systems, where each step has a corresponding compensating action if a later step fails, since no single database transaction can span all of them.

9.2 Sources consulted

The architectural guidance in this document, particularly the double-entry ledger pattern and PCI-DSS card-handling rules, draws on the following sources reviewed while preparing it:

- Exilion Technologies, "Double-Entry Accounting: The Abacus Secret to FinTech System Success" -- on the Account/Transaction table split and why double-entry underlies reliable fintech systems.

- Finlego, "How to Build a Real-Time Ledger System with Double-Entry Accounting" -- on the properties a true ledger must guarantee: balance integrity, append-only history, reversibility, and auditability.
- Payflux (Medium), "Why Your Fintech Needs a Double-Entry Ledger" -- on why a plain transactions table does not qualify as a ledger.
- ACI Worldwide, "Can cardholder data be stored without involving PCI scope?" -- on PCI DSS Requirement 3 and the approved methods for rendering PAN data unreadable.
- PCI Security Standards Council, "PCI Data Storage Do's and Don'ts" -- the official at-a-glance summary of what cardholder data may and may not be stored, referenced for the CARD table design.
- Global Payments Integrated, "PCI Rules for Storing Credit Card Numbers in a Database" -- on PAN masking display rules and the prohibition on storing Sensitive Authentication Data after authorization.

9.3 Suggested further reading

- PCI Security Standards Council official PCI DSS documentation, for the authoritative and most current version of the card data storage rules summarized in Section 2.5 and 3.6.
- Martin Kleppmann, Designing Data-Intensive Applications, particularly the chapters on transactions and consistency, for the concurrency control concepts in Section 7.4.
- Public engineering blog posts from Stripe, Square, or similar payments companies on their ledger architecture, several of which walk through the double-entry pattern in production detail beyond what a single document can cover.